

# Functions

## CMSC 323: Design and Implementation of Programming Languages

This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in *Programming Languages: Application and Interpretation*.

# Design Decisions

- ◆ Our functions will have names
- ◆ They will have a single parameter

# AE (Arithmetic Expression)

{ - 20 { + { + 10 10 }  
{ + 10 10 } } }

{ - 20 { + { + 17 17 }  
{ + 17 17 } } }

{ - 20 { + { + 3 3 }  
{ + 3 3 } } }

# WAE (With and Arithmetic Expressions)

```
{with {x 10}
  {- 20 {+ {+ x x}
            {+ x x}}}}
```

```
{with {x 17}
  {- 20 {+ {+ x x}
            {+ x x}}}}
```

```
{with {x 3}
  {- 20 {+ {+ x x}
            {+ x x}}}}
```

# F1WAE (Functions (Version 1))

```
{def fun {f x}                                     {f 10}
  {- 20 {+ {+ x x}
           {+ x x}}}}                             {f 17}

                                                    {f 3}
```

- They allow us to reuse expressions
- Note that we need to be able to define them and then call them (They have two parts)

# F1WAE

```
{deffun {f x}                                {f 10}
  {- 20 {twice
        {twice x}}}}                        {f 17}

{deffun {twice y}                             {f 3}
  {+ y y}}
```

# F1WAE

## Grammar to define the function

```
{deffun {f x}
      {- 20 {twice
             {twice x}}}}}
```

```
{f 10}
```

```
{f 17}
```

```
{deffun {twice y}
      {+ y y}}
```

```
{f 3}
```

- `deffun` defines a new form
- The first `<id>` is the name of the function
- The second `<id>` is a parameter to the function
- `<F1WAE>` represents an expression for the body of the function

```
<FunDef> ::= {deffun {<id> <id>} <F1WAE>}
```

# F1WAE

```
{deffun {f x}                                     {f 10}
  {- 20 {twice                                     {f 17}
    {twice x}}}}
{deffun {twice y}                                  {f 3}
  {+ y y}}
```

Grammar to *invoke*  
the function

<id> is the function  
name

<F1WAE> is the  
argument passed to  
the function

<FunDef> ::= {deffun {<id> <id>} <F1WAE>}

<F1WAE> ::= . . .  
| {<id> <F1WAE>}



# F1WAE Grammar

Extending our WAE to include functions

$\langle \text{FunDef} \rangle ::= \{ \text{def fun } \{ \langle \text{id} \rangle \langle \text{id} \rangle \} \langle \text{F1WAE} \rangle \}$

$\langle \text{F1WAE} \rangle ::= . . .$

$\langle \text{num} \rangle$

|  $\{ + \langle \text{F1WAE} \rangle \langle \text{F1WAE} \rangle \}$

|  $\{ - \langle \text{F1WAE} \rangle \langle \text{F1WAE} \rangle \}$

|  $\{ \text{with } \{ \langle \text{var} \rangle \langle \text{F1WAE} \rangle \} \langle \text{F1WAE} \rangle \}$

|  $\langle \text{var} \rangle$

|  $\{ \langle \text{id} \rangle \langle \text{F1WAE} \rangle \}$

# Terminology

Functions are implemented in two parts. We implement

- ◆ a way to define the function
- ◆ a way to use the function

# Terminology

## Introduction

Brings a new concept. E.g. Grammar to define the function

$\langle \text{FunDef} \rangle ::= \{ \text{deffun } \{ \langle \text{id} \rangle \ \langle \text{id} \rangle \} \ \langle \text{F1WAE} \rangle \}$

## Elimination

Uses the new concept. E.g. the grammar to evaluate a function

$\langle \text{F1WAE} \rangle ::= . \ . \ . \mid \{ \langle \text{id} \rangle \ \langle \text{F1WAE} \rangle \}$

# Terminology

Elimination is a term from programming language theory

## How to Think About It

The term "elimination" comes from logic. If you have a proof of "If  $A$  is true, then  $B$  is true" ( $A \rightarrow B$ ), and you also have a proof of  $A$ , you can **eliminate** the implication to conclude that  $B$  is true (historically known as *Modus Ponens*). In programming, when you apply a function to an argument, you are evaluating ("eliminating") the function to produce its underlying return value.

# Terminology

## Introduction

Brings a new concept. E.g. Grammar to define the function

$\langle \text{FunDef} \rangle ::= \{ \text{deffun } \{ \langle \text{id} \rangle \ \langle \text{id} \rangle \} \ \langle \text{F1WAE} \rangle \}$

## Elimination

Uses the new concept. E.g. the grammar to evaluate a function

$\langle \text{F1WAE} \rangle ::= . \ . \ . \mid \{ \langle \text{id} \rangle \ \langle \text{F1WAE} \rangle \}$

# Extending our Representation

## Introduction (Definition)

```
{def fun {f x}
  {- 20 {+ {+ x x}
           {+ x x}}}}
```

## Elimination (Application)

```
{f 10}
```

```
{f 17}
```

```
{f 3}
```

# Extending our Representation

## Introduction (Definition)

```
{deffun {f x}
  {- 20 {+ {+ x x}
           {+ x x}}}}}
```

$\langle \text{FunDef} \rangle ::= \{ \text{deffun } \{ \langle \text{id} \rangle \langle \text{id} \rangle \} \langle \text{f1WAE} \rangle \}$

# Extending our Representation

Elimination (Application)

{f 10}

{f 17}

{f 3}

$\langle \text{F1WAE} \rangle ::= \dots \mid \{ \langle \text{id} \rangle \langle \text{F1WAE} \rangle \}$



# Exercise

$\langle \text{FunDef} \rangle ::= \{ \text{deffun } \{ \langle \text{id} \rangle \langle \text{id} \rangle \}$   
 $\langle \text{F1WAE} \rangle \}$

Given the grammar to the left, is the following  
a valid statement?

$\{ \text{deffun } \{ \text{fact } n \}$   
 $\{ * n \{ \text{fact } \{ - n 1 \} \} \} \}$

$\langle \text{F1WAE} \rangle ::= . . .$   
 $\langle \text{num} \rangle$   
 $| \{ + \langle \text{F1WAE} \rangle \langle \text{F1WAE} \rangle \}$   
 $| \{ - \langle \text{F1WAE} \rangle \langle \text{F1WAE} \rangle \}$   
 $| \{ \text{with } \{ \langle \text{var} \rangle \langle \text{F1WAE} \rangle \} \langle \text{F1WAE} \rangle \}$   
 $| \langle \text{var} \rangle$   
 $| \{ \langle \text{id} \rangle \langle \text{F1WAE} \rangle \}$

# Exercise

$\langle \text{FunDef} \rangle ::= \{ \text{deffun } \{ \langle \text{id} \rangle \langle \text{id} \rangle \}$   
 $\langle \text{F1WAE} \rangle \}$

$\langle \text{F1WAE} \rangle ::= . . .$   
|  $\langle \text{num} \rangle$   
|  $\{ + \langle \text{F1WAE} \rangle \langle \text{F1WAE} \rangle \}$   
|  $\{ - \langle \text{F1WAE} \rangle \langle \text{F1WAE} \rangle \}$   
|  $\{ \text{with } \{ \langle \text{var} \rangle \langle \text{F1WAE} \rangle \} \langle \text{F1WAE} \rangle \}$   
|  $\langle \text{var} \rangle$   
|  $\{ \langle \text{id} \rangle \langle \text{F1WAE} \rangle \}$

Given the grammar to the left, is the following  
a valid statement?

```
{deffun {fact n}
  {if0 n
    1
    { * n {fact {- n 1}}}}}}
```

# Exercise

$\langle \text{FunDef} \rangle ::= \{ \text{deffun } \{ \langle \text{id} \rangle \ \langle \text{id} \rangle \} \\ \langle \text{F1WAE} \rangle \}$

Given the grammar to the left, is the following a valid statement?

$\{ \text{deffun } \{ \text{sum } a \ b \} \\ \{ + \ a \ b \} \}$

$\langle \text{F1WAE} \rangle ::= \dots$   
|  $\langle \text{num} \rangle$   
|  $\{ + \ \langle \text{F1WAE} \rangle \ \langle \text{F1WAE} \rangle \}$   
|  $\{ - \ \langle \text{F1WAE} \rangle \ \langle \text{F1WAE} \rangle \}$   
|  $\{ \text{with } \{ \langle \text{var} \rangle \ \langle \text{F1WAE} \rangle \} \ \langle \text{F1WAE} \rangle \}$   
|  $\langle \text{var} \rangle$   
|  $\{ \langle \text{id} \rangle \ \langle \text{F1WAE} \rangle \}$

# Exercise

$\langle \text{FunDef} \rangle ::= \{ \text{deffun } \{ \langle \text{id} \rangle \ \langle \text{id} \rangle \}$   
 $\langle \text{F1WAE} \rangle \}$

Given the grammar to the left, is the following  
a valid statement?

$\{ \text{deffun } \{ f \ x \}$   
 $x \}$

$\langle \text{F1WAE} \rangle ::= . . .$   
 $\langle \text{num} \rangle$   
 $| \{ + \ \langle \text{F1WAE} \rangle \ \langle \text{F1WAE} \rangle \}$   
 $| \{ - \ \langle \text{F1WAE} \rangle \ \langle \text{F1WAE} \rangle \}$   
 $| \{ \text{with } \{ \langle \text{var} \rangle \ \langle \text{F1WAE} \rangle \} \ \langle \text{F1WAE} \rangle \}$   
 $| \langle \text{var} \rangle$   
 $| \{ \langle \text{id} \rangle \ \langle \text{F1WAE} \rangle \}$

# F1WAE Datatypes

Now we extend our AST to support functions

AST for our function introduction

```
(define-type FunDef
  [deffun (name : Symbol)
         (param : Symbol)
         (body : F1WAE)])
```

AST for our function elimination

```
(define-type F1WAE
  [num (n : Number)]
  [plus (lhs : F1WAE)
        (rhs : F1WAE)]
```

. . .

```
[app (fun-name : Symbol)
     (fun-arg : F1WAE)])
```

# More Exercises

Given the F1 WAE and FunDef types above, convert the following surface syntax to their AST representations.

a. `{deffun {f x}  
x}`

c. `{deffun {twice y}  
{+ y y}}`

b. `{deffun {f x}  
{- 20 {+ {+ x x}  
{+ x x}}}}`

d. `{f 10}`

e. `{f {+ 4 5}}`

# F1WAE Interp

```
(interp : (F1WAE (Listof FunDef) -> Number))  
(define (interp fvae funDefs)  
  (type-case F1WAE fvae  
    [(num n) n]  
    [(plus lhs rhs) (+ (interp lhs funDefs)  
                        (interp rhs funDefs))]  
    . . .  
    [(app name arg) . . .]))
```

```
(test (interp (num 1) empty) 1)
```

How do we want our  
interpreter to behave?

We pass a list of functions as  
a parameter to our  
interpreter along with the  
expression to be interpreted.

# F1WAE Interp

How do we want our  
interpreter to behave?

```
(interp : (F1WAE (Listof FunDef) -> Number))  
(define (interp fvae funDefs)  
  (type-case F1WAE fvae  
    [(num n) n]  
    [(plus lhs rhs) (+ (interp lhs funDefs)  
                        (interp rhs funDefs))]  
    . . .  
    [(app name arg) . . .]))
```

```
(test (interp (plus (num 1) (num 1)) empty) 2)
```



# F1WAE Interp

How do we want our  
interpreter to behave?

```
(interp : (f1WAE (Listof FunDef) -> Number))  
(define (interp fwaefunDefs)  
  (type-case f1WAE fwaefunDefs  
    [(num n) n]  
    [(plus lhs rhs) (+ (interp lhs funDefs)  
                        (interp rhs funDefs))]  
    . . .  
    [(app name arg) . . .]))
```

```
(test (interp (plus (num 1) (num 1))  
              (list (deffun 'f 'x (plus (var 'x) (num 3))))))  
2)
```

# F1WAE Interp

How do we want our  
interpreter to behave?

```
(interp : (f1WAE (Listof FunDef) -> Number))  
(define (interp fwaefunDefs)  
  (type-case f1WAE fwaefunDefs  
    [(num n) n]  
    [(plus lhs rhs) (+ (interp lhs funDefs)  
                        (interp rhs funDefs))]  
    . . .  
    [(app name arg) . . .])))
```

```
(test (interp (app 'f (num 1))  
              (list (deffun 'f 'x (plus (var 'x) (num 3))))) 4)
```

# F1WAE Interp

How do we want our  
interpreter to behave?

```
(interp : (f1WAE (Listof FunDef) -> Number))  
(define (interp fvae funDefs)  
  (type-case f1WAE fvae  
    [(num n) n]  
    [(plus lhs rhs) (+ (interp lhs funDefs)  
                        (interp rhs funDefs))]  
    . . .  
    [(app name arg) . . .]))
```

```
(test (interp (app 'f (num 10))  
              (list (deffun 'f 'x (sub (num 20) (app 'twice (var 'x))))  
                    (deffun 'twice 'y (plus (var 'y) (var 'y))))) 0)
```

# F1WAE Interp

Evaluate the argument  
from the application of  
the function

```
(interp : (f1WAE (Listof FunDef) -> Number))  
(define (interp fvae funDefs)  
  (type-case f1WAE fvae  
    [(num n) n]  
    [(plus lhs rhs) (+ (interp lhs funDefs)  
                        (interp rhs funDefs))]  
    . . .  
    [(app name arg) . . . . .]))
```

So, before we figure out how this would work, let's backtrack...

# Some Java code

```
public class FunctionEval {  
  
    public static void main (String[] arg) {  
        int output = squareNumber(reverseDigits(89345));  
    }  
  
    public static int squareNumber(int number) {  
        return number * number;  
    }  
  
    public static int reverseDigits(int number) {  
        int reversed = 0;  
        int digit;  
        while (number != 0) {  
            digit = number % 10;           // Extract the last digit  
            reversed = reversed * 10 + digit; // Append it to the reversed number  
            number = number / 10;          // Remove the last digit  
        }  
        return reversed;  
    }  
}
```

Let's consider the function call in the main method.  
What are the steps necessary to evaluate it?

# Some Java code

```
public class FunctionEval {  
  
    public static void main (String[] arg) {  
        int output = squareNumber(reverseDigits(89345));  
    }  
  
    public static int squareNumber(int number) {  
        return number * number;  
    }  
  
    public static int reverseDigits(int number) {  
        int reversed = 0;  
        int digit;  
        while (number != 0) {  
            digit = number % 10;           // Extract the last digit  
            reversed = reversed * 10 + digit; // Append it to the reversed number  
            number = number / 10;          // Remove the last digit  
        }  
        return reversed;  
    }  
}
```

1. Evaluate reverseDigits()

2. Substitute the result in the body  
of squareNumber()  
(which means we have  
to “fetch” the body of  
squareNumber())

3. Evaluate the result of 2.

Let's consider the function call in the main method.  
What are the steps necessary to evaluate it?

# F1WAE Interp

```
(interp : (f1WAE (Listof FunDef) -> Number))  
(define (interp fvae funDefs)  
  (type-case f1WAE fvae  
    [(num n) n]  
    [(plus lhs rhs) (+ (interp lhs funDefs)  
                        (interp rhs funDefs))]  
    . . .  
    [(app name arg) . . . (interp arg funDefs) . . .])))
```

1. Evaluate the argument  
from the application of  
the function

# F1WAE Interp

```
(interp : (f1WAE (Listof FunDef) -> Number))
(define (interp fvae funDefs)
  (type-case f1WAE fvae
    [(num n) n]
    [(plus lhs rhs) (+ (interp lhs funDefs)
                        (interp rhs funDefs))]
    . . .
    [(app name arg) (local [(define func (lookup name funDefs))]
                          . . . (interp arg funDefs) . . .))])
```

Use the function name  
(name) to lookup the  
function definition in the  
list of functions (FunDefs).  
(the “fetch”)



# F1WAE Interp

```
(interp : (f1WAE (Listof FunDef) -> Number))
(define (interp fvae funDefs)
  (type-case f1WAE fvae
    [(num n) n]
    [(plus lhs rhs) (+ (interp lhs funDefs)
                        (interp rhs funDefs))]
    . . .
    [(app name arg) (local [(define func (lookup name funDefs))]
                          . . (subst (deffun-param func)
                                    (interp arg funDefs)
                                    (deffun-body func)) . . .)]))
```

Substitute the interpreted  
argument for the  
parameter name in the  
function body.

# Recall: Substitution

```
(subst : (Symbol Number WAE -> WAE))
(define (subst id val wae)
  (type-case WAE wae
    [(num n) wae]
    [(plus lhs rhs) (plus (subst id val lhs)
                           (subst id val rhs))]
    [(sub lhs rhs) (sub (subst id val lhs)
                        (subst id val rhs))]
    [(var my-id) (if (eq? id my-id) (num val) wae)]
    [(with my-id value body) (if (eq? id my-id)
                                  (with my-id (subst id val value) body)
                                  (with my-id (subst id val value)
                                              (subst id val body)))]))
```

What if the body of the  
"with" is also a "with"?

# F1WAE Interp

```
(interp : (f1WAE (Listof FunDef) -> Number))
(define (interp fvae funDefs)
  (type-case f1WAE fvae
    [(num n) n]
    [(plus lhs rhs) (+ (interp lhs funDefs)
                        (interp rhs funDefs))]
    . . .
    [(app name arg) (local [(define func (lookup name funDefs))]
                          . . (subst (deffun-param func)
                                     (interp arg funDefs)
                                     (deffun-body func)) . . .]))]
```

Substitute the interpreted  
argument for the  
parameter name in the  
function body.

```
(interp : (f1W
(define (interp
  (type-case f
    [(num n) n
    [(plus lhs
```

Evaluate the body of the  
function after the  
substitution of  
parameters for  
arguments.

# Lookup

Lookup searches for a  
function with name fun-  
name in a list of function  
definitions

```
(lookup : (Symbol (Listof FunDef) -> FunDef))  
(define (lookup fun-name funDefs)
```

# Lookup

Lookup searches for a  
function with name fun-  
name in a list of function  
definitions

```
(lookup : (Symbol (Listof FunDef) -> FunDef))  
(define (lookup fun-name funDefs)  
  (cond  
    [(empty? funDefs) (error 'lookup "Function not found")]  
    [(eq? fun-name (deffun-name (first funDefs))) (first funDefs)]  
    [else (lookup fun-name (rest funDefs))]))
```

# Subst

Expand subst from WAE to  
take care of app as well.

```
(subst : (Symbol Number f1WAE -> f1WAE))  
(define (subst id val f1wae)  
  (type-case f1WAE f1wae  
    . . .  
    [(app name arg) (app name (subst id val arg))]))
```

# Exercise

Use a hash map instead of a list as the ADT for your function definitions. See hash map functions in the slides below and here: [https://docs.racket-lang.org/plait/Predefined\\_Functions\\_and\\_Constants.html#%28part.\\_.Hash\\_.Tables%29](https://docs.racket-lang.org/plait/Predefined_Functions_and_Constants.html#%28part._.Hash_.Tables%29)



# Hash Map Functions

The make-hash function creates a mutable hash table that is initialized with a given mapping of keys to values (as a list of tuples pairing keys to values).

## Example

```
(define m-ht (make-hash (list (pair 1 "apple") (pair 2 "banana"))))
```

Note the **hash** function also makes a hash table, but it is immutable.  
So use **make-hash**.

# Hash Map Functions

The `hash-ref` function works on a hash table to find the value for a given key. If the hash table contains a mapping for a given key, `hash-ref` returns the key's value wrapped with `some`. Otherwise, `hash-ref` returns `(none)`.

## Example:

```
(define m-ht (make-hash (list (pair 1 "apple") (pair 2 "banana"))))  
(hash-ref m-ht 1) ;; returns (some apple)
```

# Hash map functions

## Example

```
(define m-ht (make-hash (list (pair 1 "apple") (pair 2 "banana"))))  
(hash-ref m-ht 3) ;; returns (none)
```

# Hash map processing

Processing some and none returned by hash-ref:

Given the following hash map:

```
(define fundefs (make-hash (list (pair 'f (deffun 'f 'x (plus (var 'x) (var 'x)))))))
```

Lookup a function using type-case as follows:

```
(type-case (Optionof FunDef) (hash-ref fundefs 'f)  
  [(some function) function]  
  [(none) (error 'lookup "function not found")])
```